1. **Define function? Differentiate between iterative and recursive function?**
   A function in C is a block of code that performs a specific task. It is designed to improve code reusability, modularity, and clarity. A function is executed when it is called from the main program or another function.

Feature	Iterative Function	Recursive Function
Definition	Uses loops (for, while) to repeat tasks	Function calls itself repeatedly
Memory Usage	Less memory (no extra stack frames)	More memory (uses call stack for each call)
Speed	Generally faster	Generally slower due to function call overhead
Termination	Terminates using loop condition	Terminates using base case
Example	Factorial using loop	Factorial using recursion

2. **Define function prototypes? Write their uses.**

   A **function prototype** is a declaration of a function that tells the compiler the function's **name**, **return type**, and **parameters**, **before its actual definition** appears in the code.
   It is usually written **before the** main() **function** or in a header file.
   return_type function_name(parameter_list);

   Uses of Function Prototypes:
   1. **Informs the compiler** about the function's return type and parameters before its actual definition.
   2. **Enables calling** functions before their definitions in the code.
   3. **Helps in type checking** by ensuring correct number and types of arguments are passed during function calls.
   4. **Improves code readability** and structure by keeping interface separate from implementation.
   5. **Supports modular programming** by allowing functions to be declared in header files.

3. **Write the advantages of using function? Describe the purpose of return statement?**
   Advantages of Using Functions:
   1. **Code Reusability:** Functions allow you to reuse code by calling the same function multiple times instead of rewriting the same logic.
   2. **Modularity:** Programs can be divided into smaller, manageable parts. Each function handles a specific task, improving organization and readability.
   3. **Ease of Debugging and Testing:** Since functions are isolated, testing and fixing errors become easier.

4. **Improved Readability:** Code becomes clearer and easier to understand when divided into named functions.
5. **Reduces Code Length:** Helps avoid repetition, reducing overall code size and complexity.

Purpose of Return Statement:

- The **return** statement is used to **exit** from a function and optionally **send back a value** to the calling function.
- It **transfers control** from the function back to where it was called.

4. Explain Call-by-Value and Call-by-Address with examples.

1. Call-by-Value: In **Call-by-Value**, a **copy** of the actual argument is passed to thefunction.
Changes made inside the function **do not affect** the original variables.
Example:

```
#include <stdio.h>
void changeValue(int x) {
    x = x + 10;
}
int main() {
    int a = 5;
    changeValue(a);
    printf("Value of a: %d", a);  // Output: 5
    return 0;
}
```

2. Call-by-Address (Call-by-Reference): In **Call-by-Address**, the **address** (pointer) of the variable is passed.
Changes made inside the function **affect the original variable**.

Example:

```
#include <stdio.h>
void changeValue(int *x) {
    *x = *x + 10;
}
int main() {
    int a = 5;
    changeValue(&a);
    printf("Value of a: %d", a);  // Output: 15
    return 0;
}
```

5. Create a C function to swap two integer data and call the function from the main() function. Don't use any third variable.

```c
#include <stdio.h>

// Function to swap two integers without using a third variable
void swap(int *a, int *b) {
    *a = *a + *b;
    *b = *a - *b;
    *a = *a - *b;
}

int main() {
    int x = 10, y = 20;
    printf("Before swapping: x = %d, y = %d\n", x, y);
    swap(&x, &y);  // Call the function by passing addresses
    printf("After swapping:  x = %d, y = %d\n", x, y);
    return 0;
}
```

6. **Create a C function that calculates factorial of a number and call it from main().**

```c
#include <stdio.h>

// Function to calculate factorial
int factorial(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        int result = factorial(num);  // Call the function
        printf("Factorial of %d is %d\n", num, result);
    }

    return 0;
}
```

7. **Create a C function to find out the real roots of a quadratic equation using user defined function quad().**

```c
#include <stdio.h>
#include <math.h>
```

```
// User-defined function to find real roots
void quad(float a, float b, float c) {
    float discriminant, root1, root2;

    discriminant = b * b - 4 * a * c;

    if (discriminant > 0) {
        root1 = (-b + sqrt(discriminant)) / (2 * a);
        root2 = (-b - sqrt(discriminant)) / (2 * a);
        printf("Two distinct real roots: %.2f and %.2f\n", root1,
root2);
    }
    else if (discriminant == 0) {
        root1 = -b / (2 * a);
        printf("Two equal real roots: %.2f and %.2f\n", root1,
root1);
    }
    else {
        printf("No real roots exist (roots are imaginary).\n");
    }
}

int main() {
    float a, b, c;

    printf("Enter coefficients a, b, and c: ");
    scanf("%f %f %f", &a, &b, &c);

    if (a == 0) {
        printf("Not a quadratic equation (a should not be 0).\n");
    } else {
        quad(a, b, c);  // Call the user-defined function
    }

    return 0;
}
```

8. **Write a program in C to compute the sum of the series 1!/1+2!/2+3!/3+4!/4+5!/5 using function.**

```
#include <stdio.h>

// Function to calculate factorial of n
int factorial(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}

int main() {
    int i;
```

```
    double sum = 0.0;

    // Calculate sum of series
    for (i = 1; i <= 5; i++) {
        sum += (double)factorial(i) / i;
    }

    printf("Sum of the series = %.2lf\n", sum);

    return 0;
}
```

9. Create a C function to convert a decimal number to a binary number using function.

```
#include <stdio.h>

// Function to convert decimal to binary and print it
void decimalToBinary(int n) {
    int binary[32];  // Array to store binary digits
    int i = 0;

    if (n == 0) {
        printf("0");
        return;
    }

    // Convert decimal to binary by repeated division
    while (n > 0) {
        binary[i] = n % 2;
        n = n / 2;
        i++;
    }

    // Print binary digits in reverse order
    for (int j = i - 1; j >= 0; j--) {
        printf("%d", binary[j]);
    }
}

int main() {
    int num;

    printf("Enter a decimal number: ");
    scanf("%d", &num);

    printf("Binary representation: ");
    decimalToBinary(num);
    printf("\n");

    return 0;
}
```

10. Explain the role of C preprocessor.

The **C preprocessor** is a tool that processes your source code **before** actual compilation. It handles instructions called **preprocessor directives**, which begin with #.

Roles and Functions of C Preprocessor:

1. **Macro Expansion:**
   It replaces macros (defined by #define) with their corresponding values or code snippets.
2. **File Inclusion:**
   It includes the contents of header files (#include), allowing reuse of code and declarations.
3. **Conditional Compilation:**
   It allows compiling code conditionally using directives like #ifdef, #ifndef, #if, #else, and #endif. This helps in making code portable and configurable.
4. **Removing Comments:**
   It removes comments from the source code before compilation.
5. **Line Control:**
   It manages line numbers and file names for better error reporting using #line.

11. Differentiate between macro and function.

Feature	Macro	Function
Definition	A macro is a preprocessor directive that defines a code snippet or constant (#define).	A function is a block of code that performs a specific task and is executed at runtime.
Processing Time	Expanded by the preprocessor before compilation (text substitution).	Executed during program runtime.
Type Checking	No type checking as it is simple text substitution.	Performs type checking during compilation.
Overhead	No function call overhead; faster execution.	Has function call overhead (stack operations).
Debugging	Difficult to debug as it has no symbol or scope.	Easier to debug with clear scope and symbols.
Usage	Used for constants or inline code snippets.	Used for reusable code blocks with logic.